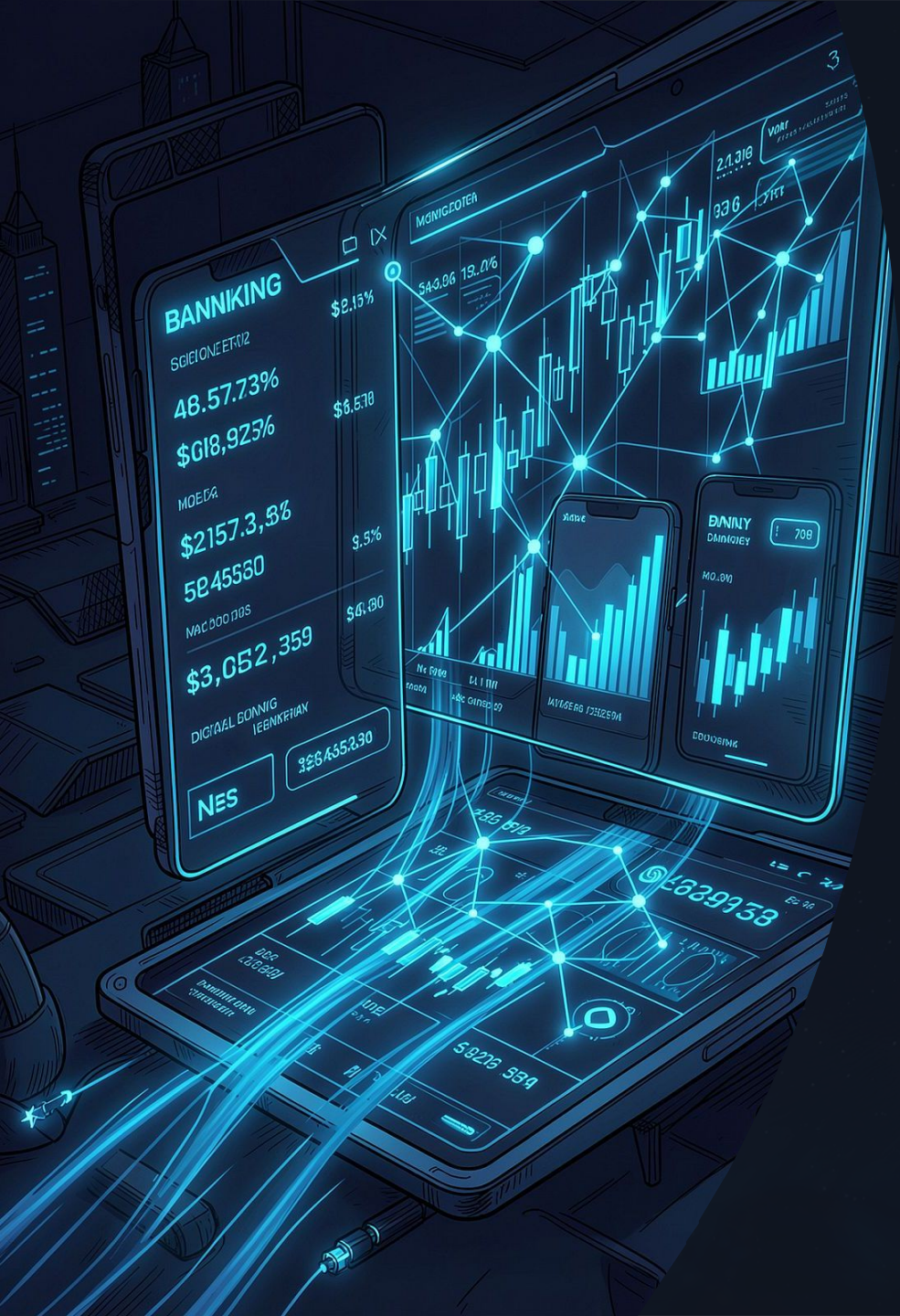
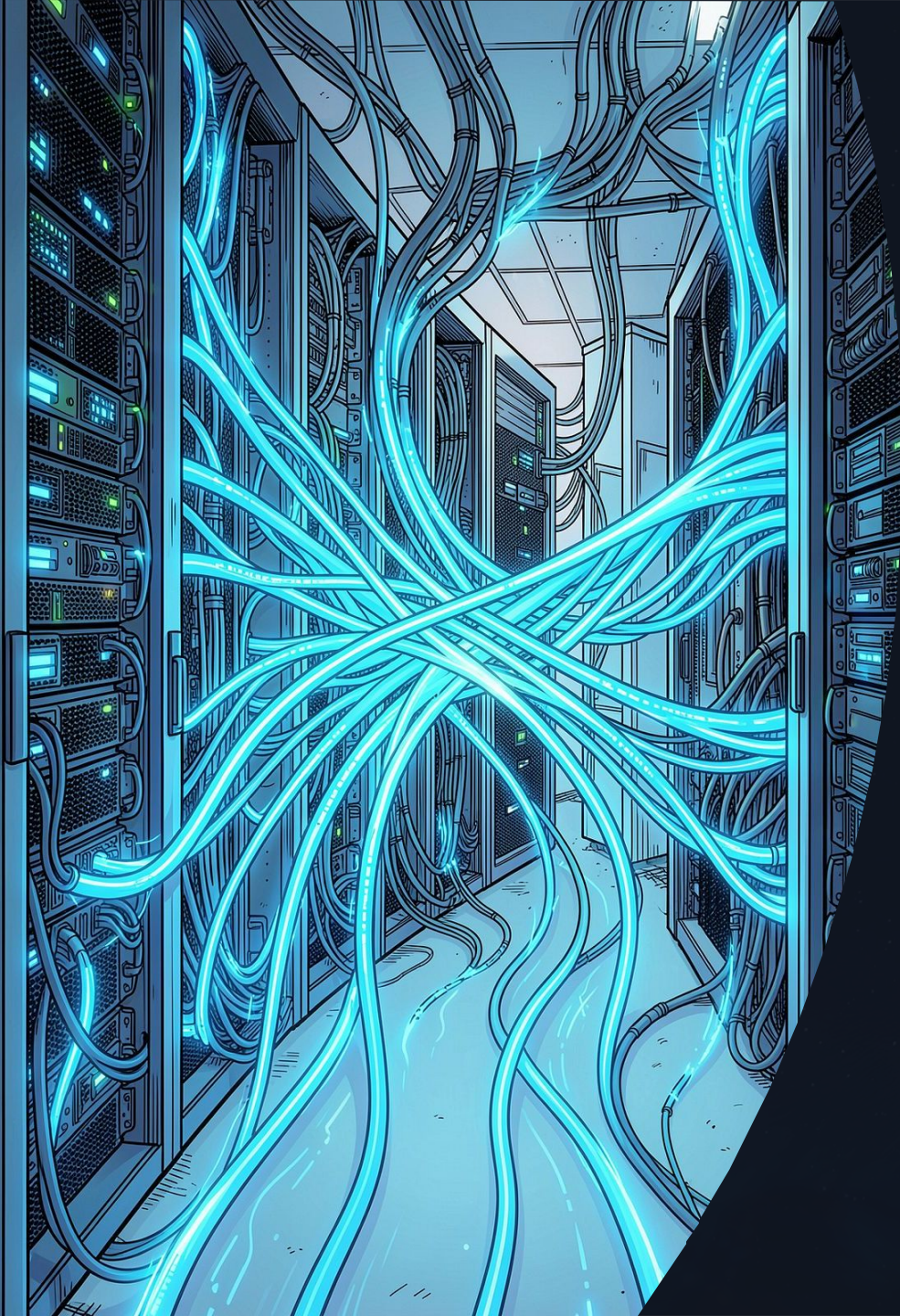




SmartBancs App

Documento Técnico de Arquitectura

Sistema de transacciones financieras en tiempo real con recomendaciones de IA — diseñado para operar bajo alta concurrencia, restricciones de core legado y latencia crítica.



El Desafío: Tres Restricciones Simultáneas

SmartBancs debe operar bajo condiciones extremas que determinan cada decisión de arquitectura.

⚡ Alta Concurrencia

Picos de hasta **10,000 TPS** que el sistema debe absorber sin degradación.

🏛️ Core Legado Bancs

El core degrada su rendimiento si recibe consultas directas a ese volumen.

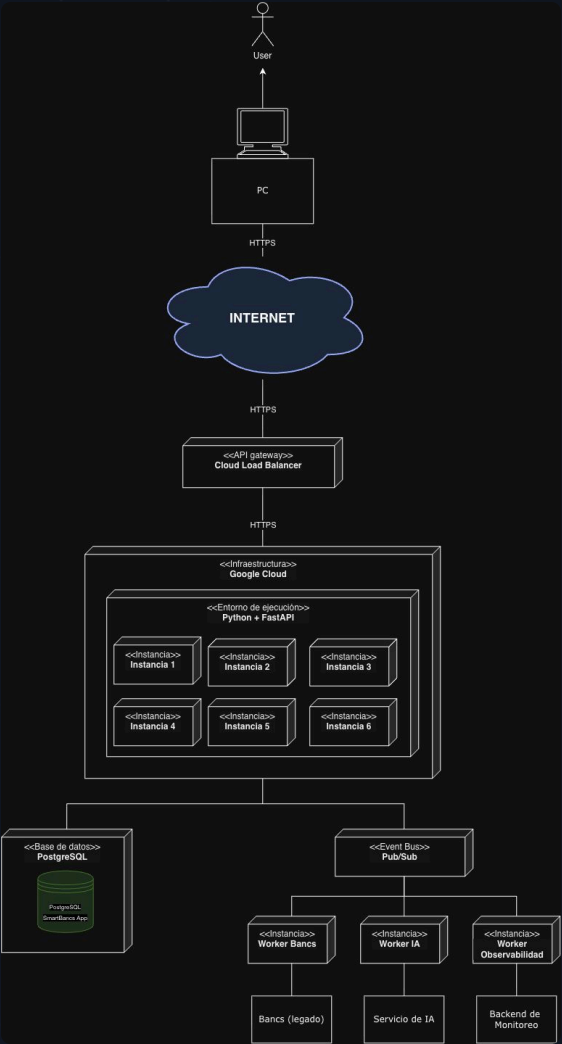
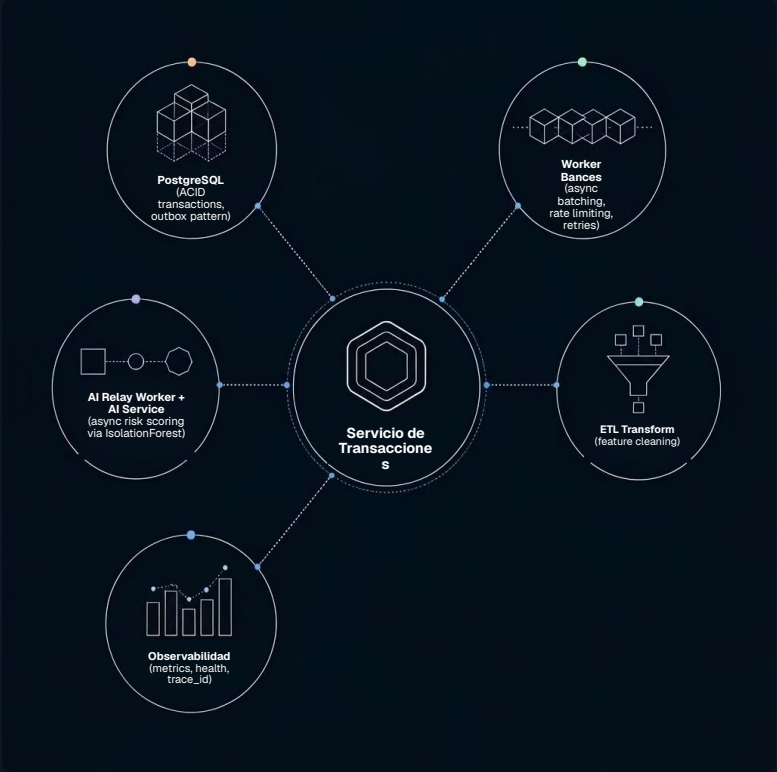
🕒 Latencia Crítica

Tiempo de respuesta **inferior a 2 segundos** para transferencias, sin bloqueo de IA.

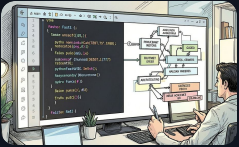
Arquitectura General del Sistema

El diseño separa el camino síncrono crítico de todas las integraciones externas mediante el patrón **Transactional Outbox**, garantizando que la latencia del endpoint principal dependa únicamente de una transacción local de base de datos.

El único camino síncrono es **transaction-service ↔ PostgreSQL**. Todo lo demás es asíncrono.

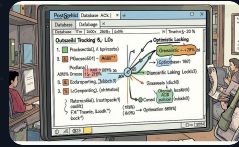


Componentes Clave



Transaction-Service

Python/FastAPI. Único componente en el camino síncrono crítico. Arquitectura hexagonal que aísla la regla de negocio de la infraestructura.



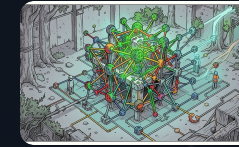
PostgreSQL + Outbox

Base de datos ACID con locking optimista. La tabla `outbox_events` desacopla las integraciones externas del camino crítico.



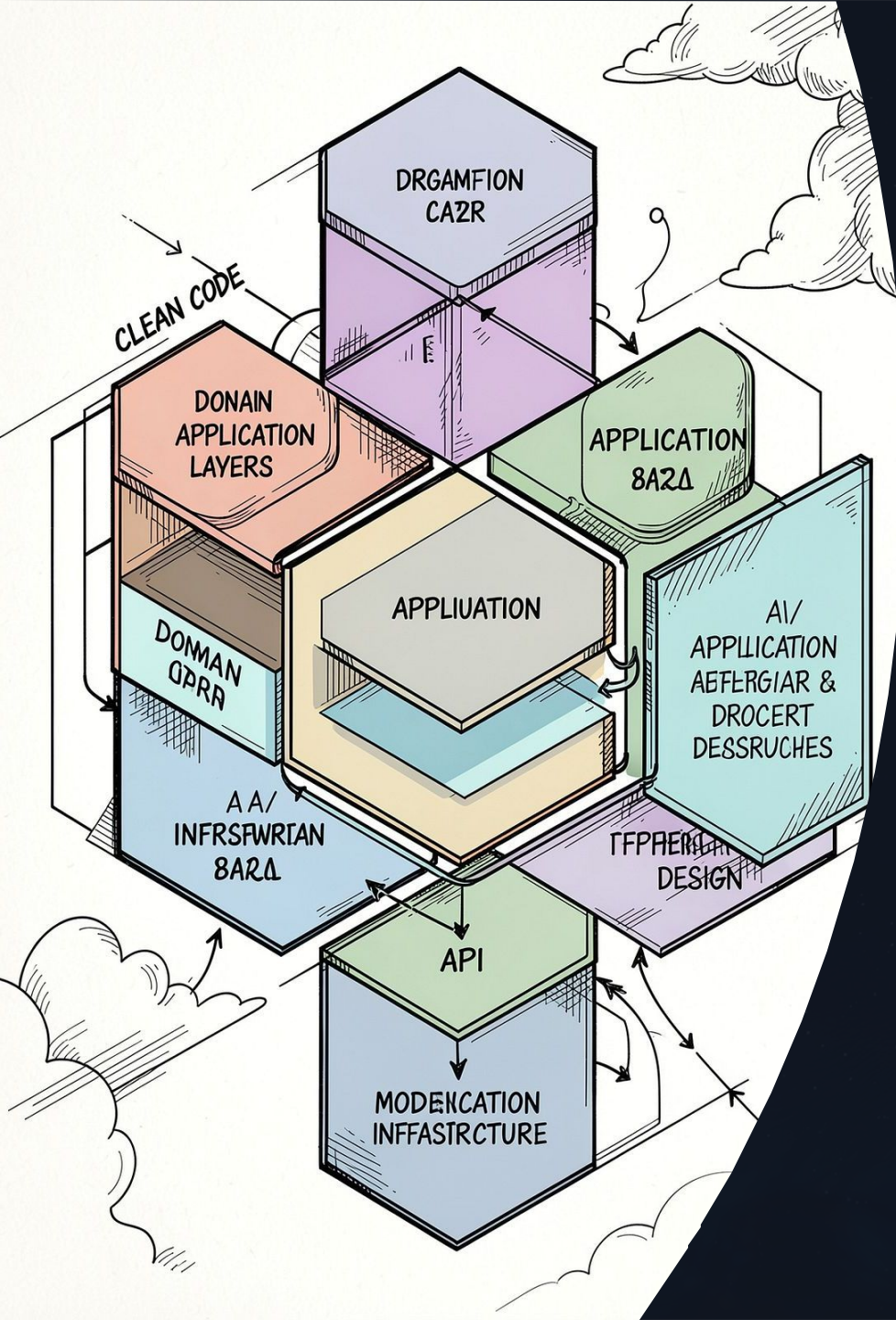
Worker Bances

Consumidor asíncrono del outbox. Aplica **batching**, **rate limiting y reintentos con backoff** para proteger el core legado.



AIRelayWorker + AI-Service

Microservicio HTTP independiente con modelo IsolationForest. Su caída **nunca afecta la latencia** porque el consumo es asíncrono.



Arquitectura Hexagonal del Transaction-Service

La regla de negocio no depende de FastAPI ni de SQLAlchemy, permitiendo testear sin dependencias externas y sustituir cualquier capa sin reescribir la lógica.

1

Domain

Entidades y excepciones puras: `Account`, `Transaction`, `OutboxEvent`.

2

Application

`ProcessTransactionUseCase` + puertos (Protocols) + DTOs.

3

Infrastructure

Adaptadores concretos: engine Postgres, modelos ORM, repositorios.

4

API

Capa HTTP: router, schemas Pydantic, middleware, manejo de errores.

Flujo del Endpoint **POST /api/transactions**

El endpoint recibe `idempotency_key`, `account_id` y `amount`, devolviendo el nuevo saldo en menos de 2 segundos.



La idempotencia garantiza que reintentos del cliente no dupliquen transacciones. El locking optimista con reintentos acotados maneja la concurrencia sin bloquear lecturas.

Rendimiento, Seguridad y Escalabilidad

Rendimiento

El camino síncrono se acota a una transacción local de BD. Worker Bancs y AIRelayWorker operan de forma asíncrona, protegiendo la latencia crítica.

Seguridad

Cada servicio valida su input. Credenciales inyectadas por variable de entorno. Logs con SQL parametrizado y truncado. Postgres restringido a red interna de Docker Compose.

Escalabilidad

Transaction-service y **ai-service** escalan horizontalmente de forma independiente según su propia demanda, sin acoplamiento entre ellos.

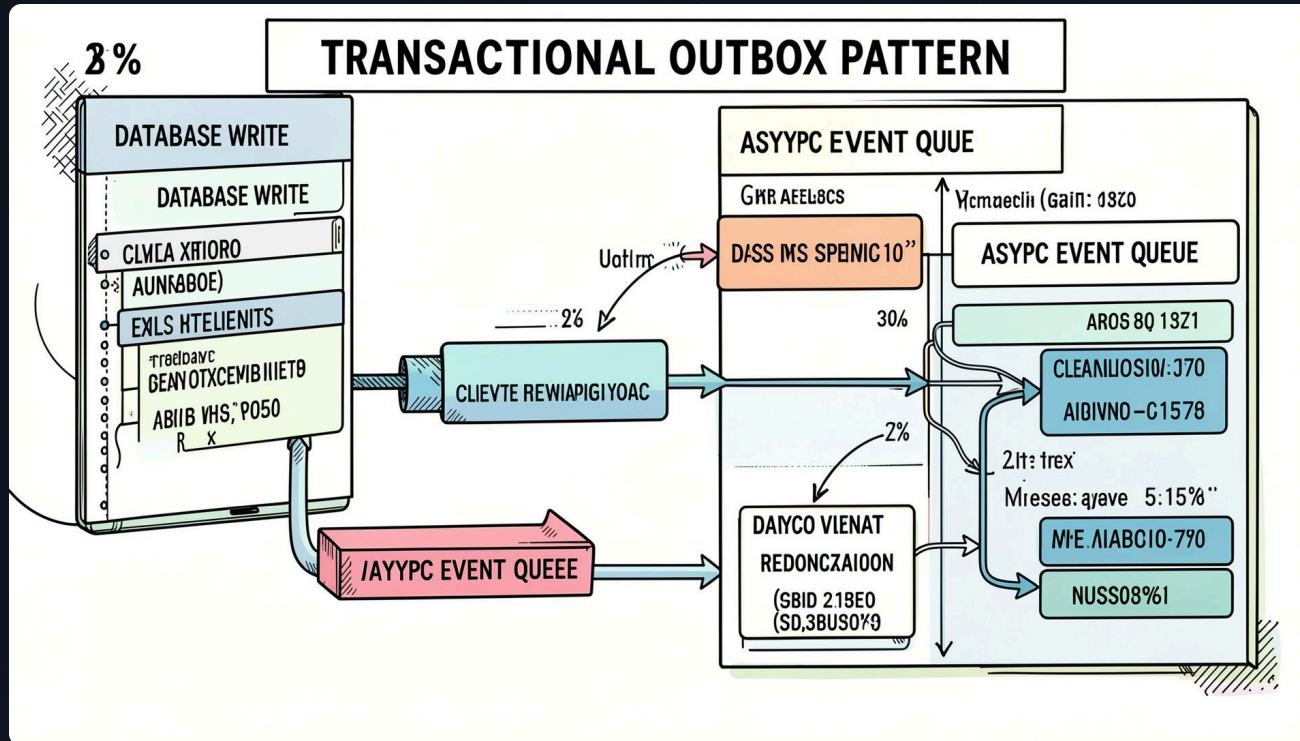
10,000 TPS

Capacidad máxima de pico

<2s

Latencia objetivo

Patrón Transactional Outbox



El patrón **Transactional Outbox** es el mecanismo central que desacopla el camino síncrono de dos integraciones de baja confiabilidad: **Bancs** y el **servicio de IA**.

→ Escritura atómica

La transacción de negocio y los eventos outbox se persisten en una sola operación ACID.

→ Polling con SKIP LOCKED

Los workers consumen eventos sin colisiones entre instancias paralelas.

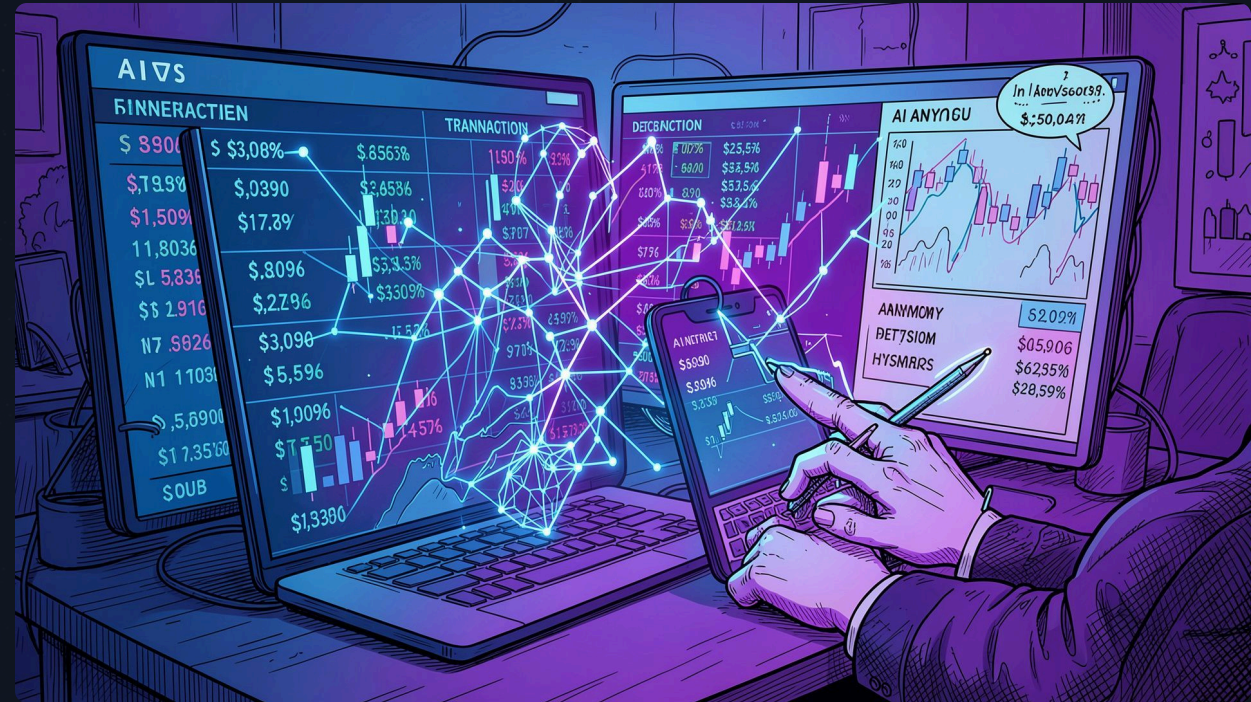
→ Resiliencia garantizada

Si Bancs o la IA fallan, los eventos se reintentan automáticamente sin pérdida de datos.

IA Asíncrona: AI Relay Worker + AI-Service

Diseño del Pipeline de Riesgo

El modelo **IsolationForest** opera de forma completamente desacoplada. El `etl/transform.py` limpia y estandariza los datos crudos, garantizando que el modelo siempre consuma features con el mismo esquema JSON Lines.



Timeout corto en HTTP

Si la IA no responde, la transacción no se ve afectada.

Risk Score en BD

El resultado se escribe en `risk_scores` para consulta posterior.

Reemplazo independiente

El modelo puede sustituirse sin tocar el `transaction-service`.



Observabilidad Transversal

Operar un sistema dimensionado para 10,000 TPS requiere visibilidad completa en tiempo real. Ambos servicios exponen métricas, health checks y logs con `trace_id` correlacionado.



Métricas Expuestas

Endpoints `/metrics` en ambos servicios para monitoreo de throughput, latencia y errores.



Health Checks

Endpoints `/health` para verificación de estado y orquestación de contenedores.



Logs + Trace ID

Cada transacción lleva un `trace_id` que correlaciona logs entre servicios para debugging distribuido.